Software Requirements Specification (SRS) & Project Plan

Version 0
Group 15
COMPSCI 4ZP6
Dr. Mehdi Moradi

Name	**Email**	**Role**
Selina Kim	kim391@mcmaster.ca	Full-stack developer, QA
Giovanna Gerada	geradag@mcmaster.ca	Program Manager, UX/UI Designer, Full-stack developer
Marwa Khafagy	khafam1@mcmaster.ca	Backend Developer, QA
Stella Gu	gus46@mcmaster.ca	Backend Developer, Data Engineer
Tinu Joseph	josept12@mcmaster.ca	Full-stack developer, UX/UI Designer

Glossary 3
Contributions 3
1. The Purpose of the Project 4
2. Client and Stakeholders 4
3. Project Constraints 5
4. Functional Requirements 6
5. Data and Metrics 8
6. Non-functional requirements 9
7. Risks 11
8. Team Meeting and Communication Plan 11
9. Team member Roles 12
10. Workflow Plan 12
11. Proof of Concept Demonstration Plan 14
12. Technology 14
13. Project Scheduling: Include a Gantt chart 15

Glossary

IDE	Integrated Development Environment
Main language	The language a client is proficient in at a high level
PR	Pull request
Target language	The language a client is learning and is not yet fully proficient in
TTS	Text-to-speech
Spaced repetition algorithm	An algorithm designed to move information into long-term memory by reviewing content at increasing optimally scheduled intervals
STT	Speech-to-text
UX/UI	User Experience and User Interface
VS Code	Visual Studio Code

Contributions

Authors	Sections
Selina Kim	1, 4, 9, 10.5, 11, 12, 13
Giovanna Gerada	2, 4, 6.1-6.4, 9, 10.5, 12, 13
Marwa Khafagy	4, 7, 9, 10.1-10.4, 10.5, 12, 13
Stella Gu	3, 4, 8, 9, 10.5, 12, 13
Tinu Joseph	4, 5, 6.5-6.8, 9, 10.5, 12, 13

1. The Purpose of the Project

1.1 Problem Statement

Language learners frequently encounter new words in their daily lives while reading, browsing social media, or conversing, but capturing and reviewing those new words effectively can become tedious. Typically, users jot unfamiliar words in memo apps or take screenshots with the intention of later adding them to a flashcard app. However, this process requires manually inputting each word and searching for its definition, which creates friction in the learning experience. As a result, many learners never return to organize or study these words, leading to lost learning opportunities and inconsistent progress. While flashcard apps exist, most require manual input, offer limited automation, or hide core features behind paywalls.

1.2 Project Objectives

The goal of the project is to:

- Develop a mobile flash card app with enhanced features that remove barriers in the language learning experience.
- Provide a frictionless user experience by automating flashcard generation based on user input.
- Provide core flashcard features for creating, viewing, organizing, and reviewing decks.
- Implement a spaced repetition algorithm to determine the frequency and order of the cards during revision to enhance long-term vocabulary retention.
- Maintain accessibility by ensuring major and critical features are not locked behind paywalls.

1.3 Proposed Solution

Our project aims to reduce as much friction in the user experience by developing a mobile application that automatically generates flashcards based on user input (either verbal or typed) in the learner's native or target language. Each card will include the translation and example sentences using the word in context. This allows users to focus their time and effort on actually learning the vocabulary in that language rather than spending their time manually creating study material. The platform will support deck building and storage, spaced repetition algorithms, user management, deck import/export functionality, and multi-language support.

2. Client and Stakeholders

As we are not currently intending to sell the product to a company, and want to keep our application free for users, our capstone team is the intended client.

Our primary stakeholders include language learners aged 10 and above who have both a main language and a target language as languages supported by the application. This product is aimed at individuals who enjoy using free mobile applications as a supplementary tool in their language learning journey, particularly for vocabulary acquisition and revision.

Other stakeholders include teachers and tutors, who may use the application in learning environments to support vocabulary teaching, spaced repetition practice, or as homework.

The application will be hosted on the App Store and Google Play Store so these platforms are also stakeholders. Owners of the open APIs we use (e.g. Google) are stakeholders as well.

3. Project Constraints

3.1 Scope Constraints

- Develop a mobile app for iOS and Android for ease of use when on the move
- The app focuses on vocabulary learning with flashcards
- Support user accounts, deck building, and storage
- Provide card revision with and without spaced repetition
- Limited number of supported languages due to time constraints
- Free to use

3.2 Off-the-Shelf Software Constraints

Quizlet

- Quizlet is one of the available flashcard apps but most of the functionality is behind a paywall, making it very hard or impossible to use without payment. Additionally, creating a set of flashcards is a time-consuming process. The purpose of our project is to create a free-to-use app that aids the process of creating cards.

Anki

- Anki is a popular flashcard app however the UI/UX is lacklustre and it also requires a significant amount of time to create a deck of flashcards. Our project aims to provide a better user experience and save the user's time by automatically generating cards.
- In addition, their mobile app costs a fee and their free website version has limited functionality. The purpose of our project is to create a free-to-use mobile app.

Other Flashcard Apps

- There are lots of software in the flashcard space but none that provide the full functionality that our project will.

3.3 Time Constraints

- Minimal functionality must be built for proof of concept demo on Nov 21, 2025.
- Finish product for the final demonstration on Mar 27, 2026.
- Final submission on April 4, 2026.

3.4 Budget Constraints

- The app should not cost more than $125 to create.
- We will avoid incurring token costs as they will not be reimbursed.

4. Functional Requirements

4.1 P0 - Proof of Concept

Frontend P0

- Frames:
    - User Authentication (Google OAuth, login, log out, sign up pages)
    - Home Page (used to access all other pages)
    - Decks (all decks for all languages)
    - Individual Deck View (all cards in one deck)
    - Deck Editing Frame (Add/Remove/Edit Cards)
    - Deck Revision (algorithm used to review cards)
    - Help Page (include information about how to use the app)
- Users will have the ability to create, view, and organize flashcards in a deck.
- Users should be able to review flashcards (flash card flip; word → translation + example)

Backend P0

- Application will support user management (creation, deletion, modify user info, etc.)
    - Implement Google OAuth for user authentification
- Given a word, the program will make a flash card with translation and relevant examples
- User should be able to review their decks and flashcards via a spaced repetition algorithm
- User should be able to create/delete/edit the generated flashcard
- Application will have bilingual translation support (card generation) for now
    - English, Korean
- Application will have TTS functionality for card revision
- Application will have a database for deck storage and user management
- Storage is offline and locally cached

4.2 P1 - Major Features

Frontend P1

- Frames:
    - Account Information Page (username, account creation date)
    - Update all deck frames to incorporate an image for nouns
    - Adding import/export to deck view
    - Settings Page (ex. adjust daily revision boundaries, TTS volume)

Backend P1

- Application will have multi-language translation support (card generation)
    - French, Mandarin, Japanese
- If a generated card is for a noun, an image of the noun should be generated for the card
- Application is secure
    - One user's decks can't be viewed by other users
    - Login information is protected
- Users will have the ability to import/export their decks to expand their deck library
- Application will have select settings fully implemented

- Storage is synced with a custom cloud server

4.5 P2 - Non-critical Features

Frontend P2

- Frames:
    - Update Account Information Page with Progress Tracking
    - Notification Frame
    - Add a dark mode option
    - Users have the option to filter cards/decks (date, language, …)
    - Add support for users to report bugs in Help Page
    - Add STT to Deck Editing Frame
- User can see their stats (e.g., words learned, words that still need revision, review/login streak)
- User gets a pop-up notification if they have not reviewed their cards in x amount of time
- User can choose to show parts of a card as needed (e.g. pronunciation, meaning, image, examples)

Backend P2

- User Progress Tracking (Store/retrieve learning statistics.)
- Sending out notifications to users if they have cards due or haven't revised in a while
- If a card is a non-noun, an image representing the word should be generated for the card
- Users will be able to share their decks via a link for viewing, with the option to add the deck to their library
- User can make a card in language A for recall in language B and choose to reverse which side they see and recall
    - Auto-generation will support both ways of use
- Application will have STT functionality for card generation (say a word, generate a card)

4.7 P3 - Future Application Features

Frontend P3

- Implement a page to browse public decks
- Users are able to share public decks with friends
- Users can search for other users

Backend P3

- Users can set their decks as public or private (default being private)
- Desktop Software support
- Allow users to create and edit collaborative decks
- Extend multi-language support to additional languages

5. Data and Metrics

5.1 User and Developer Metrics

The metrics recorded can be separated into user beneficial data and developer beneficial data, the users would benefit from data about their learning habits while developers could use engagement metrics and data on the spaced repetition algorithms function to further improve the application.

- Learning metrics:
    - Recall Accuracy: Correct Reviews/ Total Reviews, the percent of cards remembered correctly.
    - Retention Rate: Recall Accuracy over selected time spans such as days or weeks, measures effectiveness of spaced repetition algorithm.
    - Miss Rate: Percent of cards missed with a high repetition rate such as viewed without remembering 3+ times.
    - Perfection Rate: Number of reviews a card requires to be learned and not missed again.
- Algorithm Metrics
    - Interval size: The interval for when a card reappears should increase. This can be tracked to show the efficiency of the spaced algorithm.
    - Retrievability: The percentage a user can recall a word at any time. The spaced algorithm should aim to keep this at 70% or above.
    - Repeat Rate: The rate a card is being repeatedly shown, this would be used with the Miss Rate seen in Learning metrics but can also keep track of Algorithm's effectiveness of when to show missed words.
- Engagement Metrics
    - Reviews per session: Average number of cards reviewed in a user session.
    - Session Frequency: Average number of reviews per week or month of the user.
    - Review time per card: Number of seconds spent on a card.
    - Consistency rate: Number of consistent days app is used.

5.2 Stored Data

The application will store user details, card details and and other important metrics as discussed above, user metrics will be converted into a simple to view graph or data point on the user account frame.

- Card Data: The data for cards will be stored on the application side, the types of data for a card include text and images, as well as card id and deck id.
- User Data: The data for users will be stored on the application side, the types of data would be text, user id, account creation date, metric data as aforementioned in section 5.1.

5.3 Transient Data

The application will be required to deal with data that is not or will not be stored; these include API calls that generate a response that will generate data to be used but the data for the API call won't be.

- STT: The audio from speech to text will not be stored and simply used as an API call, the data (text) returned from the API will be stored on a card.
- TTS: Audio data for words on cards will be generated for aiding in pronunciation, the Audio will be stored with a card as well as the text used in the API call.
- Metadata: Data that will be used for metric purposes but will not be stored in perpetuity, examples being the queue and timer for a review session.

6. Non-functional requirements

6.1 Look and Feel Requirements

- The application should have a cute warm coloured pasta theme with a linguine mascot
- The interface should be clean, simple, modern, familiar, and intuitive for users
- The typefaces chosen must be legible, consistent and support non-roman characters
- All visual elements must follow a consistent design system (e.g. size, color, padding)
- The interface should employ responsive design elements for different screen sizes

6.2 Usability and Humanity Requirements

- The interface should be intuitive for users familiar with mobile language learning apps (e.g. Anki, Duolingo) and personal storage applications (e.g. Google Drive, Notes)
- The application should be usable by people with moderate fluency in English who are learning a secondary language we support (e.g. Korean, French, etc.)
- The application should have a help frame for additional guides on using the application
- The user should feel that our application is easier to use than competitor applications
- There should be constant feedback between the application and the user during their use of the application, especially during the card/deck generation process and deck revision
- All error messages should be clear, informative, and help the user find a solution

6.3 Performance Requirements

- The application should feel smooth for the user without excessive lag or wait times
    - The user should not have to wait more than 5 seconds for a card to be generated
- The application should generate cards and perform speech-to-text with accuracy
- The spaced repetition algorithm should perform with optimal time interval accuracy
- Deck storage must be properly synced between the backend and frontend application
- The application should be optimized to handle multiple users at the same time
- The application should allow users to have up to 50 decks, each with up to 2,000 cards
- The application should allow user to import/export their decks as CSV files for longevity

6.4 Operational and Environmental Requirements

- The user should have access to an Android or iOS mobile device with an account to access either the Google Play Store or App Store
- The user should optimally have 1GB of storage on their device (upper bound)
- The user should have internet access while using the application so storage is synced

- The user should allow microphone access if they intend to use speech-to-text
- The user should allow access to their files if they wish to download and upload decks
- The application should be integratable with open APIs such as Google Cloud APIs

6.5 Maintainability and Support Requirements

Maintainability:
The software shall be made in a modular fashion so that updating one component does not interfere with other components. The code base shall be documented according to the relevant standards and should be comprehensible to future developers. There should be unit tests or regression tests so that every update will be sure to not break any other components. The application shall be made in a manner that it will be future compatible to a reasonable expectation to iOS and Android updates.

Support:
The application should log errors which should be viewable for debugging and maintenance. There shall be updates and patches to fix any bugs or failures found in testing and usage of the application. Critical errors shall be presented in a user oriented fashion, and there shall be a section to report bugs to developers. There shall be a method of restoring progress if the application is migrated to a different device for users.

Adaptability:
The application shall be made available to both iOS and Android platforms and shall be dynamic to support different screen sizes and orientations. There shall be support for additional languages in the future without changing the core logic of the application. There shall be support for importing and exporting decks through CSV format.

6.6 Security Requirements

The application will use secure user authentication, user data and individual decks shall remain only accessible to the user unless made public. Users shall not be able to delete other user's decks; they would only be able to view them.

6.7 Cultural Requirements

The Application shall support multiple languages, the content shall be culturally neutral and inclusive. The software shall be readable and accessible for all users.

6.8 Compliance Requirements

The application shall adhere to privacy regulations such as GDPR and PIPEDA to handle user information properly. It shall meet the publishing and security standards required by the Google Play store for android applications. All objects and third-party resources shall follow copyright and licensing requirements. The application shall follow OWASP to adhere to having a secure and compliant software.

7. Risks

1. A significant risk stems from our dependency on APIs. Our reliance on Google Cloud APIs for functions such as translation or speech-to-text puts us at an external risk of service outages or unexpected cost increases.
2. When using APIs for example sentences, there's a risk that some sentences may originate from copyrighted work (e.g. books). Displaying these sentences without proper permission could constitute a copyright violation. Similarly, if we generate or fetch images to illustrate flashcards, we need to confirm the usage rights for those images.
    a. Possible Mitigation: Use APIs that explicitly allow educational/commercial reuse
3. Machine translation models can sometimes reinforce harmful stereotypes (e.g making gendered assumptions about roles). Additionally, example sentences may inadvertently include references that are politically or culturally inappropriate.
    a. Possible Mitigation: Allow user reporting of problematic flashcards.
4. A lot of API providers have a terms of service that may restrict how the output is being used

8. Team Meeting and Communication Plan

8.1 Team Meeting Plan

The team agrees to have regular meetings as needed for project work, sprint planning, and work sessions. Meetings will be held online or in-person, depending on everyone's availability. Frequency is expected to range between one to three times a week. Scheduling of meetings will be discussed in a prior meeting or offline with all members. When the meeting is in person, a meeting room will be booked in advance, if possible. When the meeting is online, it will be held on Discord. Meeting notes will be taken for each meeting and added to the meetings folder on our GitHub.

8.2 Team Communication Plan

The team will communicate regularly using Discord and members are expected to be responsive. We created a Discord server so that conversations can be separated into different channels such as meeting planning, Jira sprint, ideas, and general conversations.

We are using Jira for project management. Deliverables, required tasks, and any new issues will be tracked and assigned on Jira. Team members can check the Jira board and keep up to date with what everyone else is working on.

8.3 Document Sharing

The team has a shared folder in Google Drive to store our documents and will use Google Docs to work on documentation together. Meeting notes will be created on GitHub and team members can update the notes with their own as needed.

9. Team member Roles

Author	Role	Responsibilities
Selina Kim	Full-stack developer, QA	Frontend development, backend testing
Giovanna Gerada	Program Manager, UX/UI Designer, Full-stack developer	Managing Jira, app design (UX/UI), frontend development/testing, algorithm development
Marwa Khafagy	Backend Developer, QA	Backend development, frontend testing, algorithm development
Stella Gu	Backend Developer, Data Engineer	Database engineering, backend development
Tinu Joseph	Full-stack developer, UX/UI Designer	App design (UX/UI), custom storage solution development, backend testing

10. Workflow Plan

10.1 Version control

We will use GitHub as our primary version control and collaboration platform. Our workflow will follow the *Git Feature Branch* model:

- A `main` branch will represent the stable, production-ready code.
- A `develop` branch will serve as the integration branch for completed features.
- Each new feature or bug assigned by a Jira ticket will have the ticket number followed with a short explanation of the ticket in the branch name (e.g., `CP-0/translation-api-integration`)
- Code changes will be merged into `develop` via pull requests (PRs). PRs will require one peer review before approval.

10.2 Agile Methods

We have started to follow an agile scrum method using Jira for sprint planning and backlog management.

- The project will be broken into 2 week sprints.
- At the start of each sprint, tasks will be created with story points and assigned in the Jira board

- We will hold a couple of weekly stand-ups/check-ins to discuss our progress and any blockers

10.3 Data Storage

For vocabulary decks, user accounts, and spaced repetition metadata, we will use a database hosted on a cloud server and local cache.

10.4 Compute-Heavy Tasks

Currently, we do not plan to have compute-heavy tasks.

10.5 What tool/method is used to achieve each of the requirements and achieve the performance metrics that were proposed in your SRS?

Achieving Requirements

- Frontend:
    - App UX/UI design - Figma
    - Implementing App frontend - React Native, Expo
    - Code maintainability, improve development/debugging experience - Typescript
    - HTTP/HTTPS client to interact with backend API calls - Fetch API
    - Robust unit test suite to maintain code quality - Jest
- Backend:
    - Spaced repetition revision algorithm - Python
    - Users will have the ability to import/export their decks - Python
    - Users must be authenticated - Python, PostgreSQL
    - Users should be able to create/delete/edit the generated flashcard - Python, PostgreSQL
    - APIs - TTS, card generation (translations, definitions, example sentences, images) - Flask, Google Cloud APIs, Unsplash, Merriam-Webster Dictionary API
    - Storing account & specific user information, cards & decks - PostgreSQL, Storage Solution
    - Database Engine - PostgreSQL
    - Deployment and Containerization - Docker
    - Operating environment for Storage Server - Linux
    - Robust unit test suite to maintain code quality - Pytest
    - Code maintainability, improve development/debugging experience - Pydantic
    - Secure user authentication and authorization - Flask-JWT-Extended

Performance Requirements

- Frontend:
    - User interactions with the frontend interface should have immediate responses with no visible lag - React Native, Expo

- Backend:
    - API calls related to card/deck/user management should take <5 seconds - Python, Flask, Linux, external APIs
    - Cached local storage and Cloud database should sync whenever card/deck/user information is modified - Python, Flask, Linux
    - Limit the number of requests a user can make in a time period - Flask-Limiter

11. Proof of Concept Demonstration Plan

One of the main risks we can expect in the development of this project is the majority of our core features depending on internal and external APIs. It is imperative that we have a solid foundation in our database and backend server early on, such that we have less deterrents in the development process moving forward. If we do not establish a robust backend, we may not be able to demonstrate one of the critical features of our project (e.g. Card Generation API, Spaced repetition algorithm, etc.) that provided the motivation for this project in the first place.

For the proof of concept demonstration video (due on Nov. 21), we are expecting to have a working prototype of our app that ensures most of our P0 requirements are met. This includes:

- A limited working version of our database and backend server
    - Basic user authentication APIs (i.e. login, logout)
    - Card Generation API: given a word, the API returns the translation or definition of the word
    - Spaced Repetition Algorithm: will be used in determining the order and frequency in the queue of cards to display when the user is reviewing their card deck
    - Database can perform basic operations related to card decks and user management (i.e. creation, deletion, modification, retrieval of cards/decks/users)
- UX/UI designs completed on Figma and partially implemented in the frontend code

12. Technology

12.1 Frontend

We will be using React Native and Expo in Typescript as the main framework for our frontend. We will also be using Jest for testing and Fetch API for the HTTP client, both of which are already shipped by React Native. Our UX/UI designs will be done on Figma, to make it easier for collaboration.

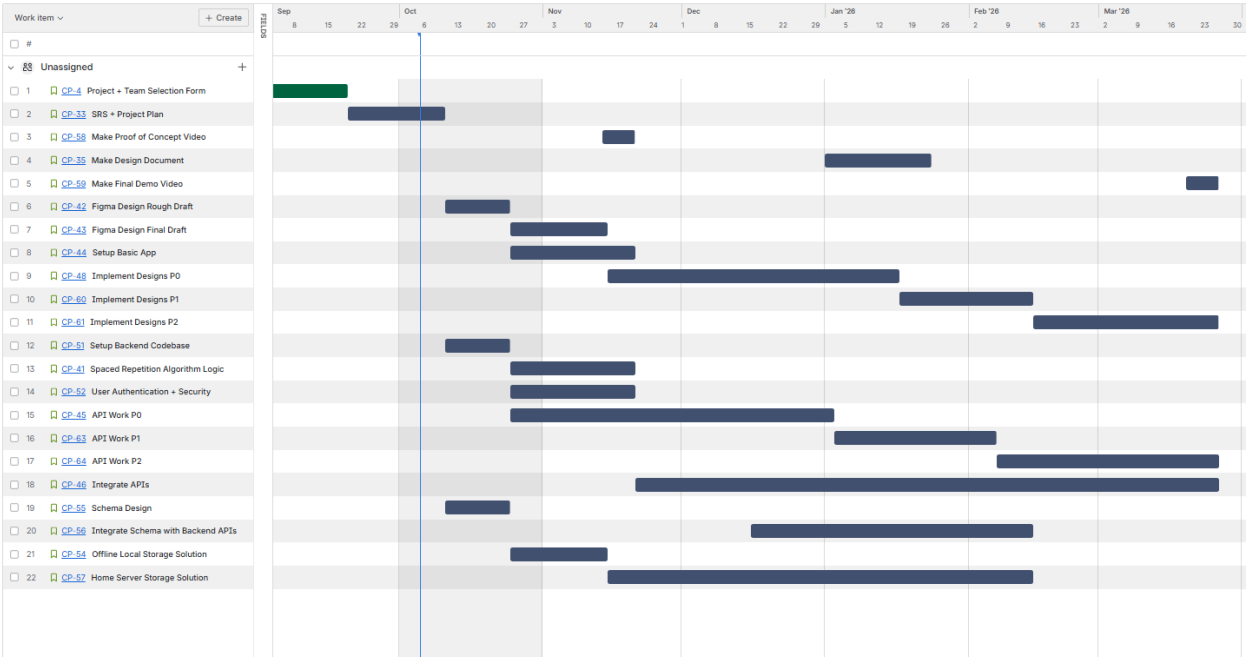
12.2 Backend

Our backend language will be Python with Poetry as the package manager and Pytest for testing. We will be using Flask as the API framework and PostgreSQL for our database. Storage will be

hosted on a home server using docker to containerize PostgreSQL, which will allow it to function as a private cloud storage solution for the backend and database.

12.3 Development Environment/Workflow Tools

We plan to use VS Code as our IDE for both frontend and backend to ease code generation through its syntax highlighting and extensions. We will use Jira as our project management tool via sprints and a backlog. We will share code via our shared GitHub repository, where we will review each other's work before we merge code into develop and main branches using Git.

13. Project Scheduling: Include a Gantt chart

We have planned the scheduling of our project according to the Gantt chart given below: